

```
library(naniar)
```

```
## Warning: package 'naniar' was built under R version 4.3.3
```

```
library(nortest)
```

```
library(mice)
```

```
## Warning in check_dep_version(): ABI version mismatch:
```

```
## lme4 was built with Matrix ABI version 1
```

```
## Current Matrix ABI version is 0
```

```
## Please re-install lme4 from source or restore original 'Matrix' package
```

```
##
```

```
## Attaching package: 'mice'
```

```
## The following object is masked from 'package:stats':
```

```
##
```

```
##      filter
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      cbind, rbind
```

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Warning: package 'ggplot2' was built under R version 4.3.3
```

```
## Loading required package: lattice
```

```
## Warning: package 'lattice' was built under R version 4.3.3
```

```
library(EnvStats)
```

```
## Warning: package 'EnvStats' was built under R version 4.3.3
```

```
## Registered S3 method overwritten by 'EnvStats':
```

```
##      method      from
```

```
##      print.estimate lava
```

```
##
```

```
## Attaching package: 'EnvStats'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      predict, predict.lm
```

```
library(psych)
```

```
## Warning: package 'psych' was built under R version 4.3.3
```

```
##
```

```
## Attaching package: 'psych'
```

```
## The following objects are masked from 'package:ggplot2':
```

```
##
```

```
##      %+%, alpha
```

```
library(car)
```

```
## Warning: package 'car' was built under R version 4.3.3
```

```
## Loading required package: carData
##
## Attaching package: 'car'
## The following object is masked from 'package:psych':
##
##     logit
## The following object is masked from 'package:EnvStats':
##
##     qqPlot
library(tseries)

## Registered S3 method overwritten by 'quantmod':
##   method      from
## as.zoo.data.frame zoo
library(forecast)
library(fastDummies)

## Warning: package 'fastDummies' was built under R version 4.3.3
library(Metrics)

##
## Attaching package: 'Metrics'
## The following object is masked from 'package:forecast':
##
##     accuracy
## The following objects are masked from 'package:caret':
##
##     precision, recall
library(tictoc)

## Warning: package 'tictoc' was built under R version 4.3.3
# Set the working directory
setwd("C:/Ngoc/TimeSeries")
```

The Beijing PM2.5 Dataset on UCI Machine Learning Repository (<https://archive.ics.uci.edu/dataset/381/beijing+pm2+5+data>) is an air quality time-series dataset collected from Beijing, containing 43824 hourly observations of PM2.5 concentration along with related weather features from 2010 to 2014. The dataset includes 12 variables:

- year: year of observation
- month: month of observation
- day: day of observation
- hour: hour of observation
- pm2.5: PM2.5 concentration (micrograms/m³)
- DEWP: Dew Point (degrees Celsius)
- TEMP: Temperature (degrees Celsius)
- PRES: Pressure (hPa)

- cbwd: Combined wind direction
- Iws: Cumulated wind speed (m/s)
- Is: Cumulated hours of snow
- Ir: Cumulated hours of rain

PM2.5 stands for *particulate matter with a diameter of 2.5 micrometers or smaller*, which is a type of air pollutant.

In this project, ARIMAX (which is an extension of the ARIMA model by including external variables) is used to forecast PM2.5 concentration over time based on past observations and external meteorological variables.

```
set.seed(10)
```

```
df <- read.csv("beijing_pm25.csv")
head(df)
```

```
##   No year month day hour pm2.5 DEWP TEMP PRES cbwd  Iws Is Ir
## 1  1 2010     1   1   0    NA  -21  -11 1021   NW  1.79 0 0
## 2  2 2010     1   1   1    NA  -21  -12 1020   NW  4.92 0 0
## 3  3 2010     1   1   2    NA  -21  -11 1019   NW  6.71 0 0
## 4  4 2010     1   1   3    NA  -21  -14 1019   NW  9.84 0 0
## 5  5 2010     1   1   4    NA  -20  -12 1018   NW 12.97 0 0
## 6  6 2010     1   1   5    NA  -19  -10 1017   NW 16.10 0 0
```

```
# Rename column
```

```
names(df)[names(df) == "pm2.5"] <- "pm25"
```

```
# Change all columns to lowercase
```

```
names(df) <- tolower(names(df))
```

```
# Drop the No column
```

```
df$no <- NULL
```

```
# Convert a categorical variable into factor
```

```
df$cbwd <- as.factor(df$cbwd)
```

```
# Summary
```

```
summary(df[5:12])
```

```
##      pm25      dewp      temp      pres      cbwd
## Min.   : 0.00  Min.   :-40.000  Min.   :-19.00  Min.   : 991  cv: 9387
## 1st Qu.: 29.00  1st Qu.: -10.000  1st Qu.:  2.00  1st Qu.:1008  NE: 4997
## Median : 72.00  Median :  2.000  Median : 14.00  Median :1016  NW:14150
## Mean   : 98.61  Mean   :  1.817  Mean   : 12.45  Mean   :1016  SE:15290
## 3rd Qu.:137.00  3rd Qu.: 15.000  3rd Qu.: 23.00  3rd Qu.:1025
## Max.   :994.00  Max.    : 28.000  Max.    : 42.00  Max.    :1046
## NA's   :2067
##      iws      is      ir
## Min.   : 0.45  Min.   : 0.00000  Min.   : 0.0000
## 1st Qu.: 1.79  1st Qu.: 0.00000  1st Qu.: 0.0000
## Median : 5.37  Median : 0.00000  Median : 0.0000
## Mean   : 23.89  Mean   : 0.05273  Mean   : 0.1949
## 3rd Qu.: 21.91  3rd Qu.: 0.00000  3rd Qu.: 0.0000
## Max.   :585.60  Max.    :27.00000  Max.    :36.0000
##
```

From the summary,

- The mean of PM2.5 concentration (*pm25*) being higher than the median suggests a right-skewed distribution. Moreover, the 3rd quartile being much lower than the maximum shows that this right-skewness is possibly due to some exceptionally high pollution values. In addition, *pm25* contains 2067 missing values, indicating the need for missing data handling.
- Dew point (*dewp*) ranges from -40 to 28 degrees Celsius, while temperature (*temp*) ranges from -19 to 42 degrees Celsius, indicating considerable temperature variation. Additionally, both *dewp* and *temp* have relatively close mean and median, suggesting that these variables are not strongly skewed. Therefore, their wide ranges are likely due to seasonal changes rather than extreme values.
- Atmospheric pressure (*pres*) is relatively stable, with values concentrated around the mean of 1016 hPa.
- For the combined wind direction (*cbwd*), southeast (SE) and northwest (NW) directions occur most frequently, while calm and variable (cv) is less common and northeast (NE) appear least frequently.
- The mean of cumulated wind speed (*iws*) being higher than the median suggests a right-skewed distribution. Furthermore, the 3rd quartile being much lower than the maximum shows that this right-skewness is possibly due to some exceptionally high wind speed values.
- The cumulated hours of snow (*is*) and rain (*ir*) are heavily concentrated at zero, indicating that snowfall and rainfall events are rare during the observation period.

```
str(df)
```

```
## 'data.frame':    43824 obs. of  12 variables:
## $ year : int  2010 2010 2010 2010 2010 2010 2010 2010 2010 2010 2010 ...
## $ month: int   1  1  1  1  1  1  1  1  1  1  1 ...
## $ day  : int   1  1  1  1  1  1  1  1  1  1  1 ...
## $ hour : int   0  1  2  3  4  5  6  7  8  9 ...
## $ pm25 : int  NA NA NA NA NA NA NA NA NA NA NA ...
## $ dewp : int  -21 -21 -21 -21 -20 -19 -19 -19 -19 -20 ...
## $ temp : num  -11 -12 -11 -14 -12 -10 -9 -9 -9 -8 ...
## $ pres : num  1021 1020 1019 1019 1018 ...
## $ cbwd : Factor w/ 4 levels "cv","NE","NW",...: 3 3 3 3 3 3 3 3 3 3 ...
## $ iws  : num   1.79 4.92 6.71 9.84 12.97 ...
## $ is   : int    0  0  0  0  0  0  0  0  0  0 ...
## $ ir   : int    0  0  0  0  0  0  0  0  0  0 ...
```

Calendar variables (year, month, day, hour) should stay numeric for time-series modeling.

```
# Check if the dataset is sorted in ascending time order or not (needed for train - test split)
all(with(df, order(year, month, day, hour) == seq_len(nrow(df))))
```

```
## [1] TRUE
```

```
head(df)
```

```
##   year month day hour pm25 dewp temp pres cbwd  iws is ir
## 1 2010     1   1   0   NA  -21  -11 1021   NW   1.79  0  0
## 2 2010     1   1   1   NA  -21  -12 1020   NW   4.92  0  0
## 3 2010     1   1   2   NA  -21  -11 1019   NW   6.71  0  0
## 4 2010     1   1   3   NA  -21  -14 1019   NW   9.84  0  0
## 5 2010     1   1   4   NA  -20  -12 1018   NW  12.97  0  0
## 6 2010     1   1   5   NA  -19  -10 1017   NW  16.10  0  0
```

```
# Split train - test as 80:20
n <- nrow(df)
split_point <- floor(0.8 * n)
```

```
train <- df[1:split_point,]
test  <- df[(split_point + 1):n,]
```

In time-series modeling, instead of random train - test split, chronological train - test split is used to preserve the time order.

```
# Check missing values
colSums(is.na(train))
```

```
## year month  day  hour pm25 dewp temp pres cbwd iws is ir
##      0      0    0    0 1968    0    0    0    0    0  0  0
```

```
# Use Little's MCAR test to check if data is MCAR or not
mcar_test(train)
```

```
## # A tibble: 1 x 4
##   statistic    df p.value missing.patterns
##   <dbl> <dbl>   <dbl>         <int>
## 1     558.   11      0             2
```

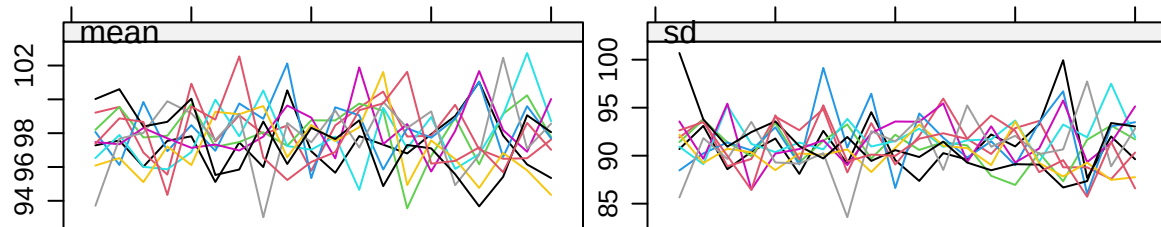
Since the p-value of Little's MCAR test is extremely small (rounded to 0), the null hypothesis can be rejected at the 0.01 significance level. This indicates that the missing data pattern is either MAR or MNAR. However, MNAR is difficult to identify and handle in practice. Therefore, unless there is strong evidence suggesting the data are MNAR, the missing data pattern is assumed to be MAR.

Based on this assumption, it is valid to use MICE imputation for the missing values of pm25.

```
combined <- rbind(train, test)
```

```
# MICE with Predictive Mean Matching (PMM) multiple imputation
imp <- mice(combined, seed = 10, method = "pmm", m = 10, maxit = 20, print = FALSE,
            ignore = c(rep(FALSE, nrow(train)), rep(TRUE, nrow(test))))
# Rows with ignore set to TRUE don't influence the parameters of the imputation model,
# but are still imputed

# Check algorithm convergence
plot(imp)
```



Iteration

The graphs of the mean and standard deviation of the imputed values across 10 iterations fluctuate a bit but mostly hover around a stable band. There's no strong upward / downward pattern. From this, it can be concluded that the algorithm has most likely converged.

```
# Full datasets
comp <- complete(imp, action = 10) # action <= m in mice()

train_comp <- comp[1:nrow(train),]
test_comp <- comp[(nrow(train)+1):nrow(combined),]

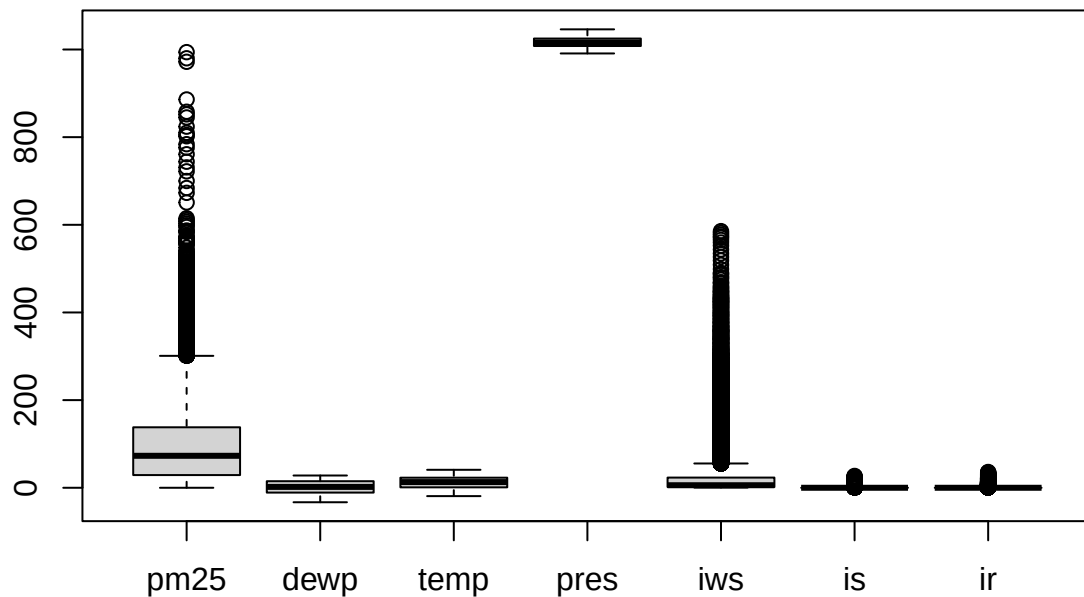
# Re-check missing values
colSums(is.na(train_comp))[colSums(is.na(train_comp))>0]

## named numeric(0)

colSums(is.na(test_comp))[colSums(is.na(test_comp))>0]

## named numeric(0)

# Check outliers of numeric variables (aside from calendar variables)
cols <- setdiff(5:12, 9)
boxplot(train_comp[cols])
```

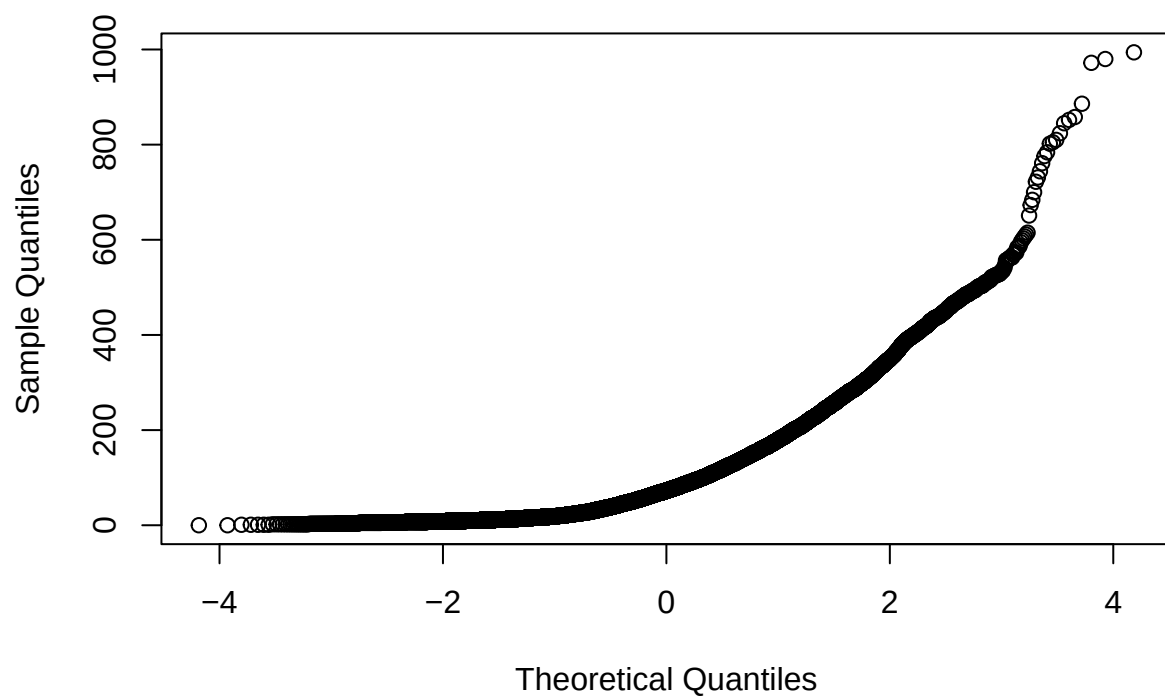


The boxplots indicate that:

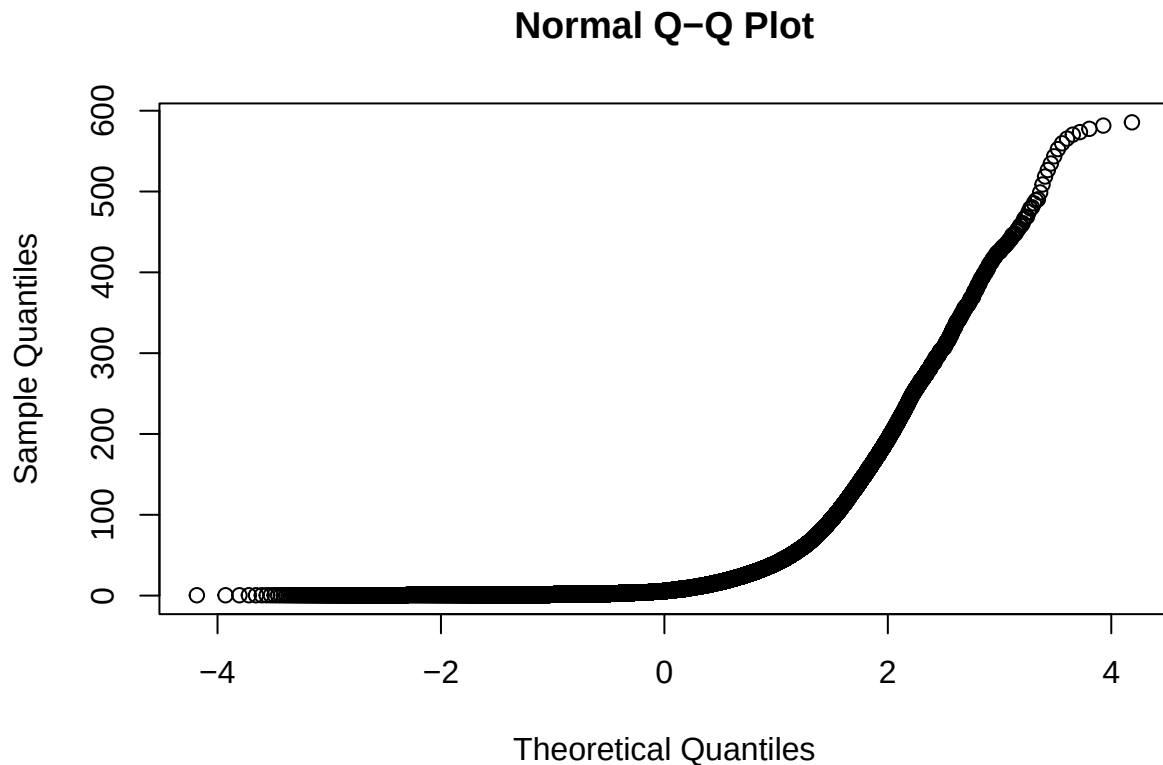
- Both PM2.5 concentration (*pm25*) and cumulated wind speed (*iws*) exhibit substantial right-skewness and contain a large number of high-value boxplot outliers, suggesting the presence of exceptionally high pollution observations and wind speed values.
- The cumulated hours of snow (*is*) and rain (*ir*) also contain some high-value boxplot outliers. However, most observations are concentrated near zero, indicating that snowfall and rainfall events are relatively rare during the observation period. Therefore, these high-value outliers are likely to be meaningful observations rather than problematic anomalies, and thus are retained in the analysis.

```
# Check normality of pm25 and iws
# QQ plot
qqnorm(train_comp$pm25)
```

Normal Q-Q Plot



```
qqnorm(train_comp$iws)
```



```
# Anderson-Darling test  
ad.test(train_comp$pm25)
```

```
##  
## Anderson-Darling normality test  
##  
## data: train_comp$pm25  
## A = 1437.1, p-value < 2.2e-16
```

```
ad.test(train_comp$iws)
```

```
##  
## Anderson-Darling normality test  
##  
## data: train_comp$iws  
## A = 5778.4, p-value < 2.2e-16
```

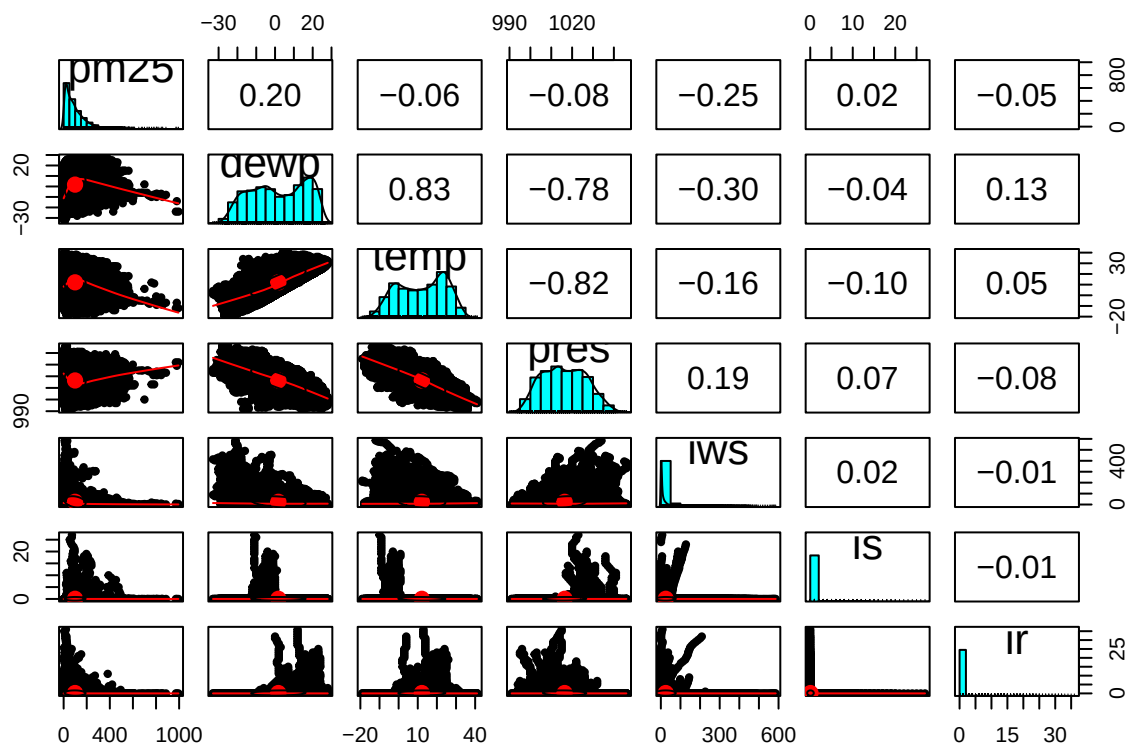
Since the p-values of Anderson-Darling normality tests are extremely small ($< 2.2 \times 10^{-16}$), the null hypothesis can be rejected at the 0.01 significance level. This indicates that *pm25* and *iws* aren't normally distributed.

Moreover, the Q-Q plots show continuous curvature across a large portion of the distributions instead of only a few isolated tail points deviating from a straight line. This suggests that the non-normality of *pm25* and *iws* is most likely not caused solely by outliers.

The Rosner test for outliers only works under the assumption that the variable being tested follows a normal distribution in the absence of outliers, and the number of outliers shouldn't exceed 10. Therefore, this outlier test isn't suitable for *pm25* and *iws*, which likely contain large numbers of outliers and aren't normally distributed even after excluding the outliers.

Due to the large number of outliers in *pm25* and *iws*, in-depth environmental knowledge is required to determine whether these observations are problematic. Hence, these values are retained in the analysis for the time being.

```
# Correlation plot (only numeric features that aren't calendar variables)
pairs.panels(train_comp[cols])
```



According to the correlation plot,

- PM2.5 concentration (*pm25*) has weak correlations with temperature (*temp*), pressure (*pres*), cumulated hours of snow (*is*) and rain (*ir*). The negative correlation between PM2.5 concentration and cumulated wind speed (*iws*) suggests that higher wind speeds may help disperse air pollutants.
- Dew point (*dewp*) and temperature (*temp*) exhibit a strong positive correlation, while temperature (*temp*) and pressure (*pres*), as well as dew point (*dewp*) and pressure (*pres*) exhibit strong negative correlations. These relationships indicate multicollinearity among these predictor variables.

For time-series modeling with ARIMA, only past values of the target variable and past forecast errors are needed. Therefore, checking for multicollinearity with VIF is not needed for ARIMA. However, ARIMAX incorporates external predictor variables into the ARIMA model, which makes it more regression - like and therefore requires no strong multicollinearity between predictors.

```
# Muticollinearity checking (only numeric features that aren't calendar variables)
model <- lm(pm25 ~ dewp + temp + pres + iws + is + ir, data = train_comp)
vif(model)
```

```
##      dewp      temp      pres      iws      is      ir
## 4.003599 4.615874 3.418183 1.140661 1.017692 1.030243
```

According to the VIF result, *dewp*, *temp* and *pres* have moderate multicollinearity ($3 < \text{VIF} < 5$), which is

to be expected because dew point and temperature are meteorologically related. However, all VIF values remain below 5, indicating that severe multicollinearity is not present. Thus, these variables are retained in the analysis.

```
# Standardize training set then apply to test set
# (only standardize numeric features that are not calendar variables)
train_sub <- setdiff(names(train_comp), c("year", "month", "day", "hour", "pm25", "cbwd"))
test_sub <- setdiff(names(test_comp), c("year", "month", "day", "hour", "pm25", "cbwd"))

train_scaled <- train_comp
train_scaled[, train_sub] <- scale(train_comp[, train_sub])

train_mean <- attr(scale(train_comp[, train_sub]), "scaled:center")
train_std <- attr(scale(train_comp[, train_sub]), "scaled:scale")

test_scaled <- test_comp
test_scaled[, test_sub] <- scale(test_comp[, test_sub], center = train_mean, scale = train_std)
```

Standardization is only applied to numeric external predictor variables.

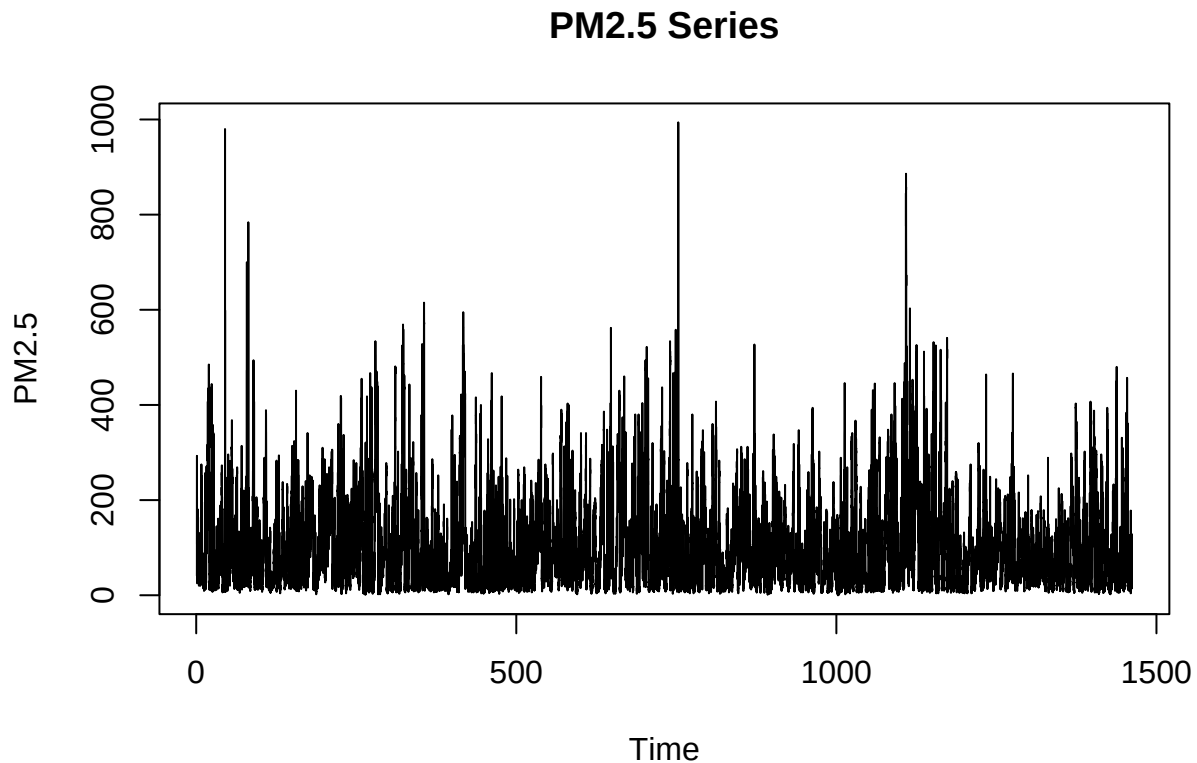
```
# Estimate the dominant seasonal period of a time series
findfrequency(train_scaled$pm25)

## [1] 26

# Convert pm25 into a time series object
train_scaled$pm25 <- ts(train_scaled$pm25, frequency = 24)
# frequency = 24 treats pm25 as a time series with each season containing 24 observations,
# since hourly data may have daily seasonality and 24 hours = 1 day

test_scaled$pm25 <- ts(test_scaled$pm25, frequency = 24)

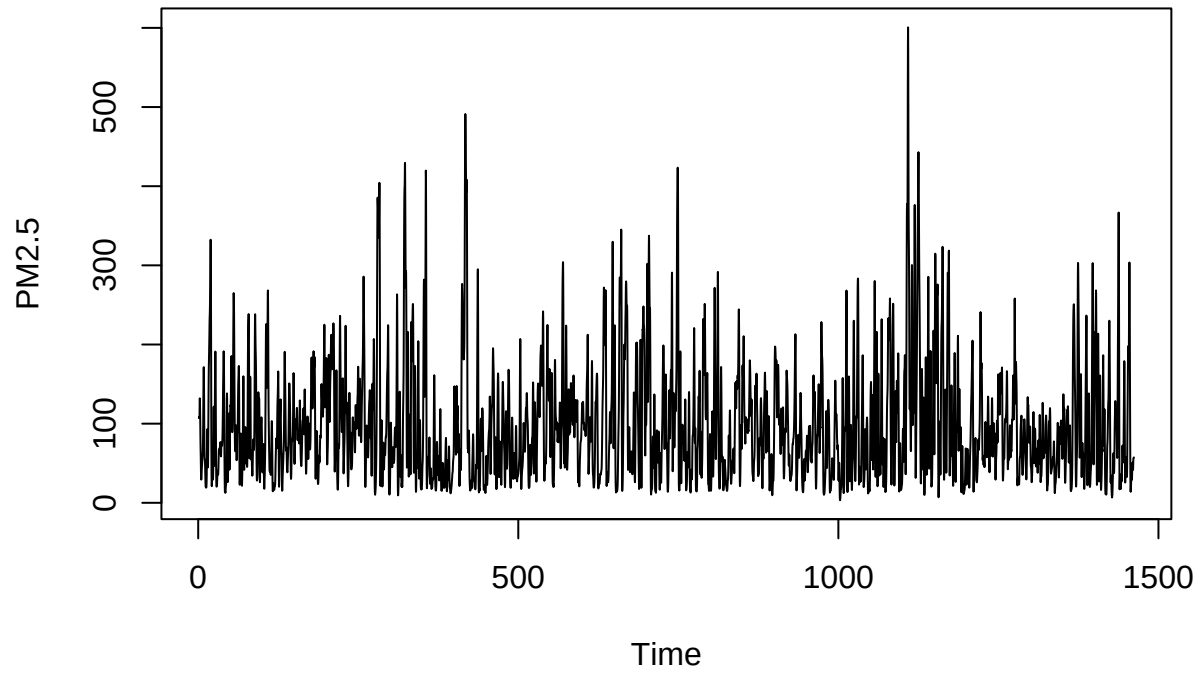
# Plot pm25 over time (Time = days)
plot(train_scaled$pm25, xlab = "Time", ylab = "PM2.5", main = "PM2.5 Series")
```



The time-series plot shows that PM2.5 concentration fluctuates substantially over time, with several sharp spikes indicating periods of exceptionally high air pollution. Although no clear upward or downward trend is observed, periods of elevated PM2.5 concentrations appear repeatedly across different years, suggesting possible seasonal or cyclical patterns. Furthermore, the mean and variance of the series don't appear constant over time, which indicates that the series may be non-stationary.

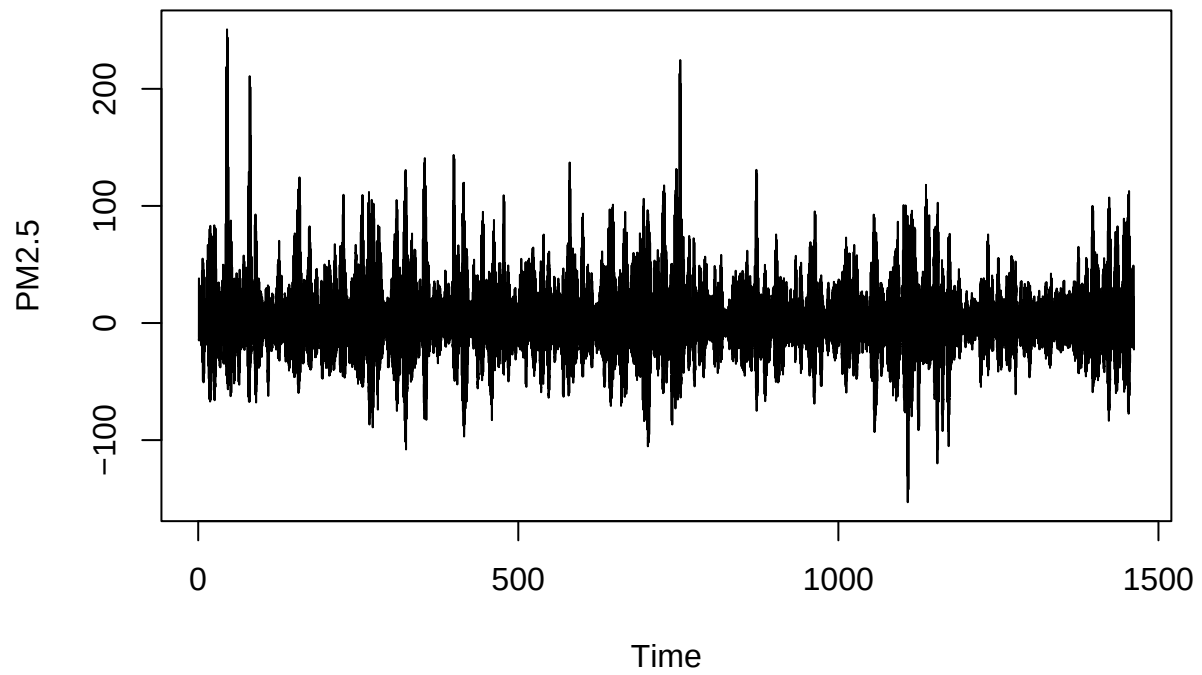
```
# Check seasonality, trend, residuals by time series decomposition
decomp <- stl(train_scaled$pm25, s.window = 7) # Window size needs to be an odd number and >= 7
plot(decomp$time.series[, "trend"], ylab = "PM2.5", main = "Trend")
```

Trend

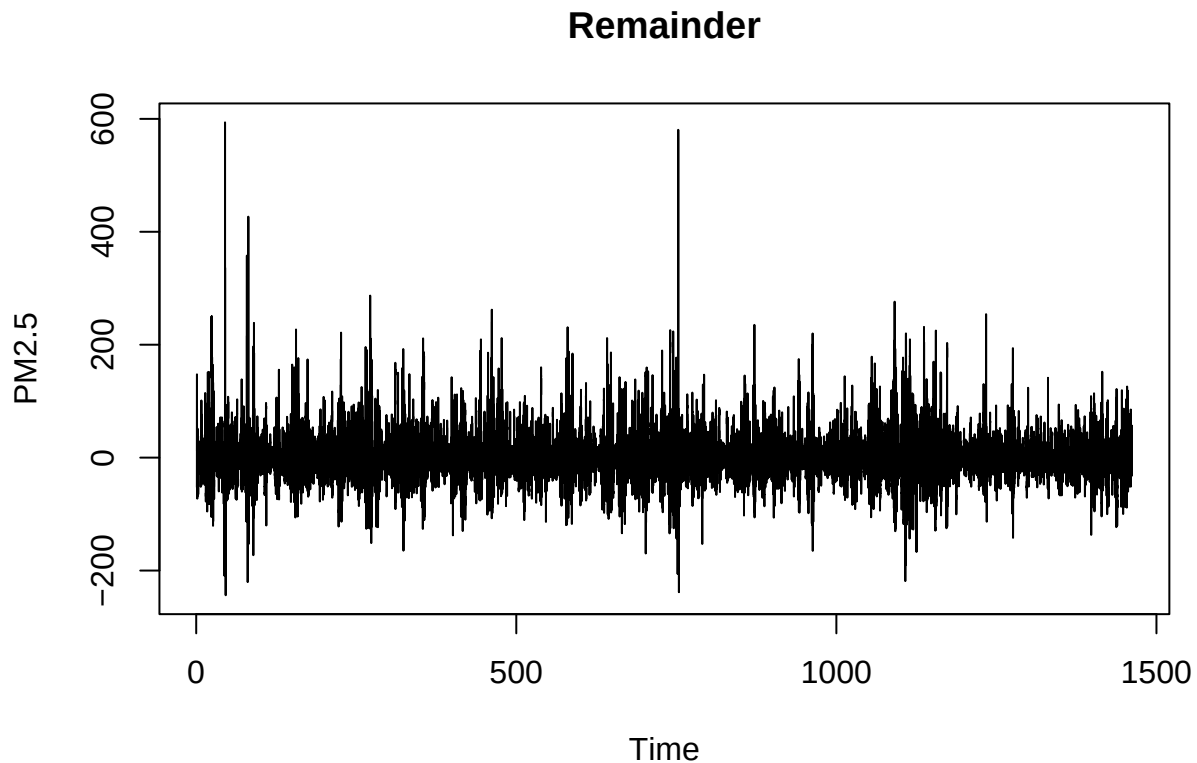


```
plot(decomp$time.series[, "seasonal"], ylab = "PM2.5", main = "Seasonal")
```

Seasonal



```
plot(decomp$time.series[, "remainder"], ylab = "PM2.5", main = "Remainder")
```



*# If you suspect there are multiple seasonal periods, use `msts()` instead of `ts()` to specify
all frequencies and use `mstl()` instead of `stl()` to get multiple seasonal plots*

From the time-series decomposition, there doesn't seem to be any upward / downward pattern in the trend plot, nor any clear daily seasonal pattern in the seasonal plot. Moreover, the remainder (residuals) plot is mostly centered around 0 and has no visible pattern, somewhat resembling white noise and suggesting that the trend and seasonal components capture most of the underlying data structure.

However, the remainder plot still displays non-constant variance and several large spikes, which may indicate the presence of outliers or short-term events not explained by the trend and seasonal components.

Check stationarity

ADF test

```
adf.test(train_scaled$pm25)
```

```
## Warning in adf.test(train_scaled$pm25): p-value smaller than printed p-value
```

```
##
```

```
## Augmented Dickey-Fuller Test
```

```
##
```

```
## data: train_scaled$pm25
```

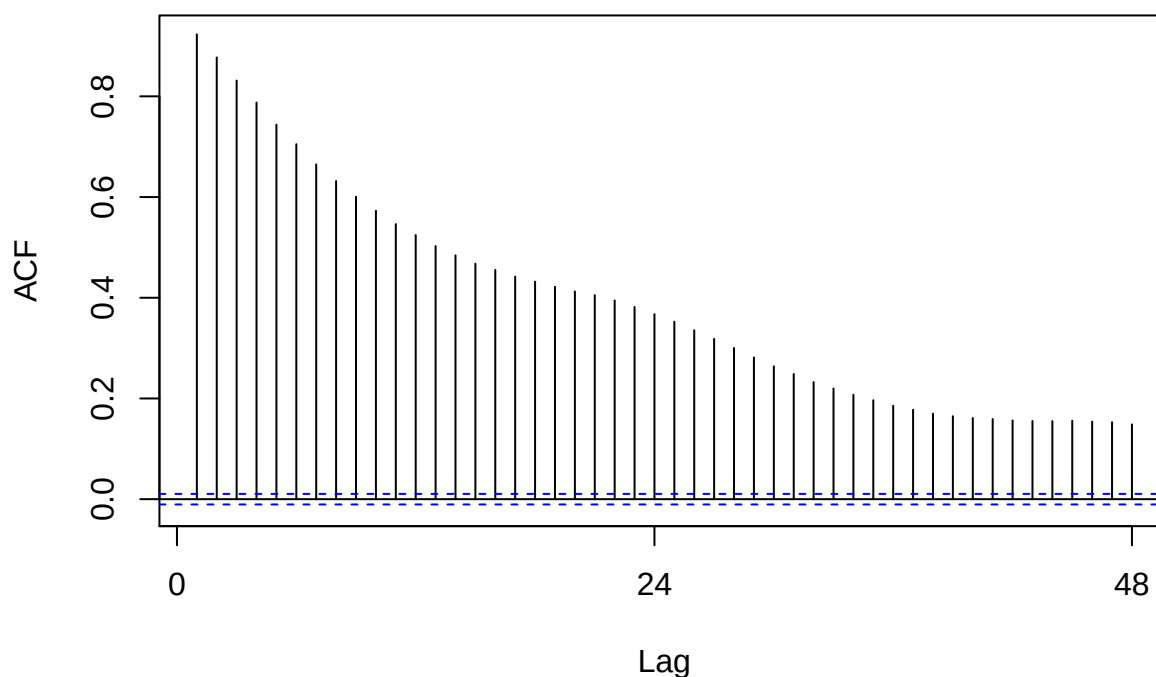
```
## Dickey-Fuller = -21.835, Lag order = 32, p-value = 0.01
```

```
## alternative hypothesis: stationary
```

Plot ACF (x axis is shown in seasons, which means 1.0 on axis = 24 lags)

```
Acf(train_scaled$pm25)
```

Series train_scaled\$pm25

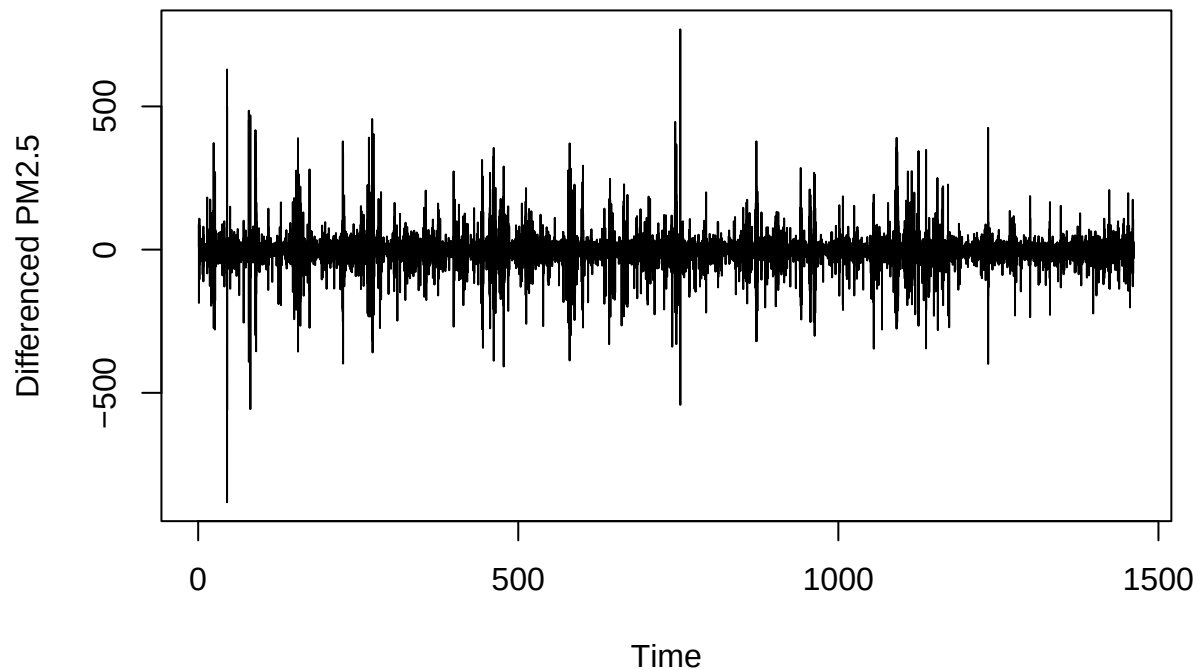


Since the p-value of the ADF test is smaller than 0.01, the null hypothesis can be rejected at the 0.01 significance level. This indicates that the time-series is stationary. However, according to the ACF plot, the ACF values drop to 0 rather slowly (slow decay), suggesting non-stationarity. Therefore, differencing (subtracting the previous observation from the current observation) on the time series will be used.

```
# 1st order differencing to remove non-stationarity (via d=1 in ARIMA/ARIMAX)
diff_pm25 <- diff(train_scaled$pm25, differences = 1)

# Plot differenced pm25 over time
plot(diff_pm25, xlab = "Time", ylab = "Differenced PM2.5", main = "Differenced PM2.5 Series")
```

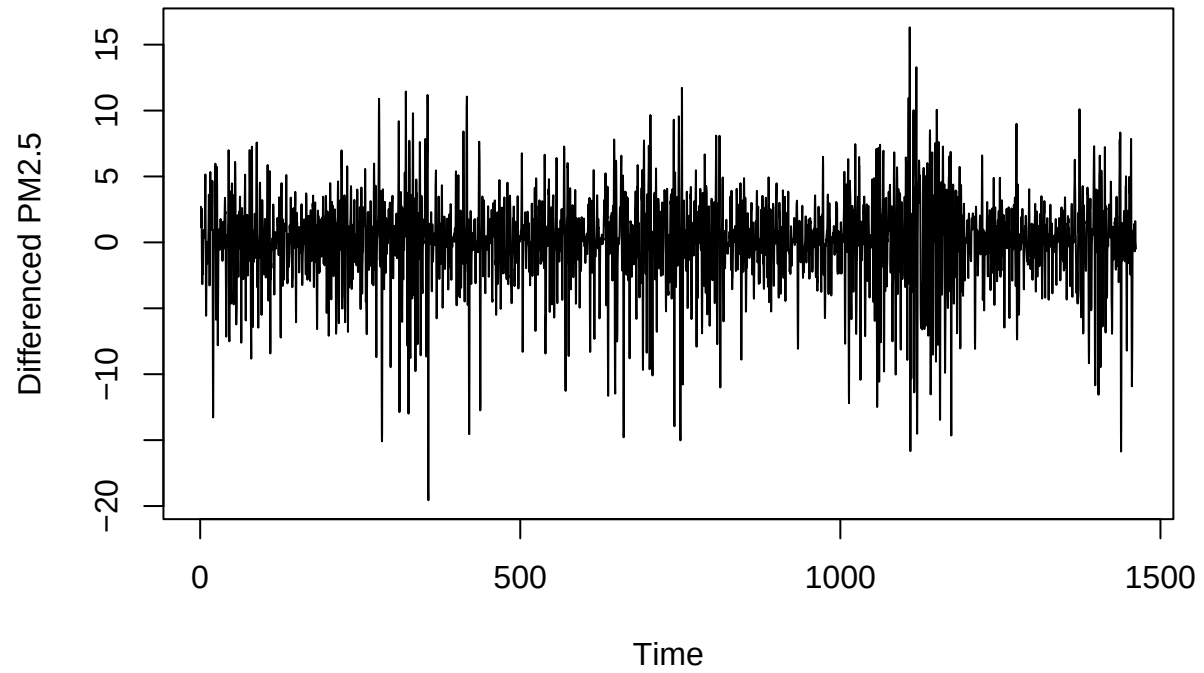
Differenced PM2.5 Series



The differenced time series plot shows that the PM2.5 concentration fluctuates around a constant mean near 0 over time, suggesting improved stationarity after first-order differencing. However, several large spikes are still present, indicating that non-constant variance and abrupt changes remain in the series. Overall, the differenced series appears more stationary and therefore more suitable for ARIMAX modeling.

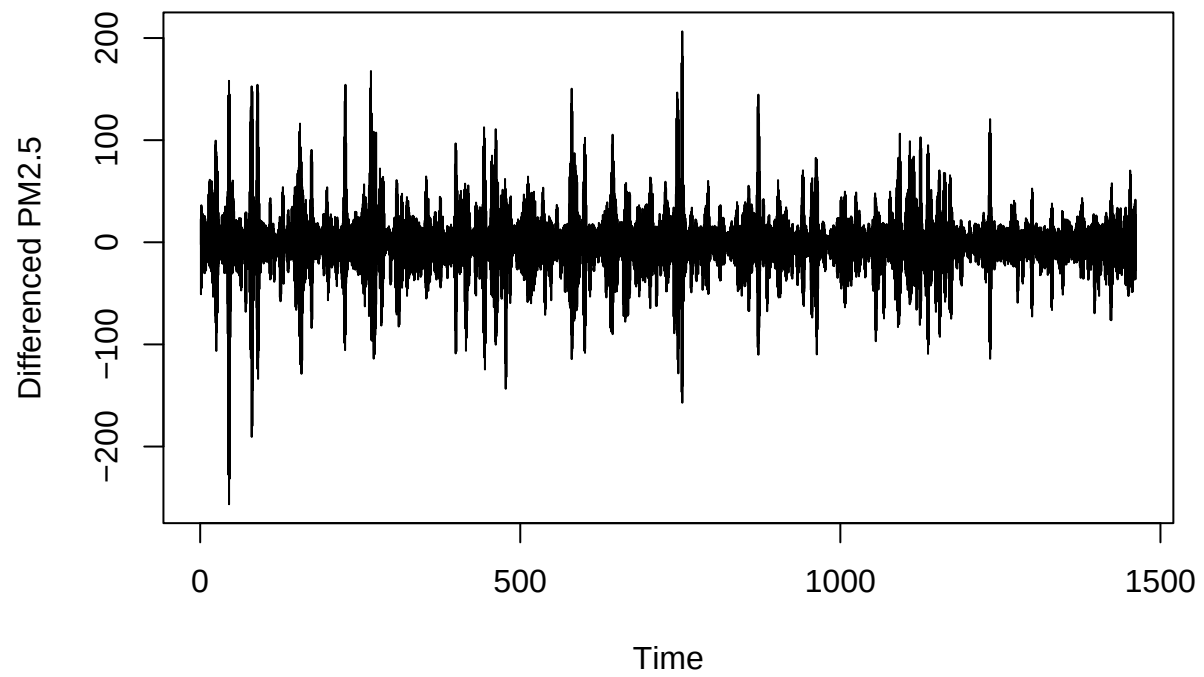
```
# Re-check seasonality, trend, residuals by time series multiple decomposition  
diff_decomp <- stl(diff_pm25, s.window = 7)  
plot(diff_decomp$time.series[, "trend"], ylab = "Differenced PM2.5", main = "Trend")
```

Trend

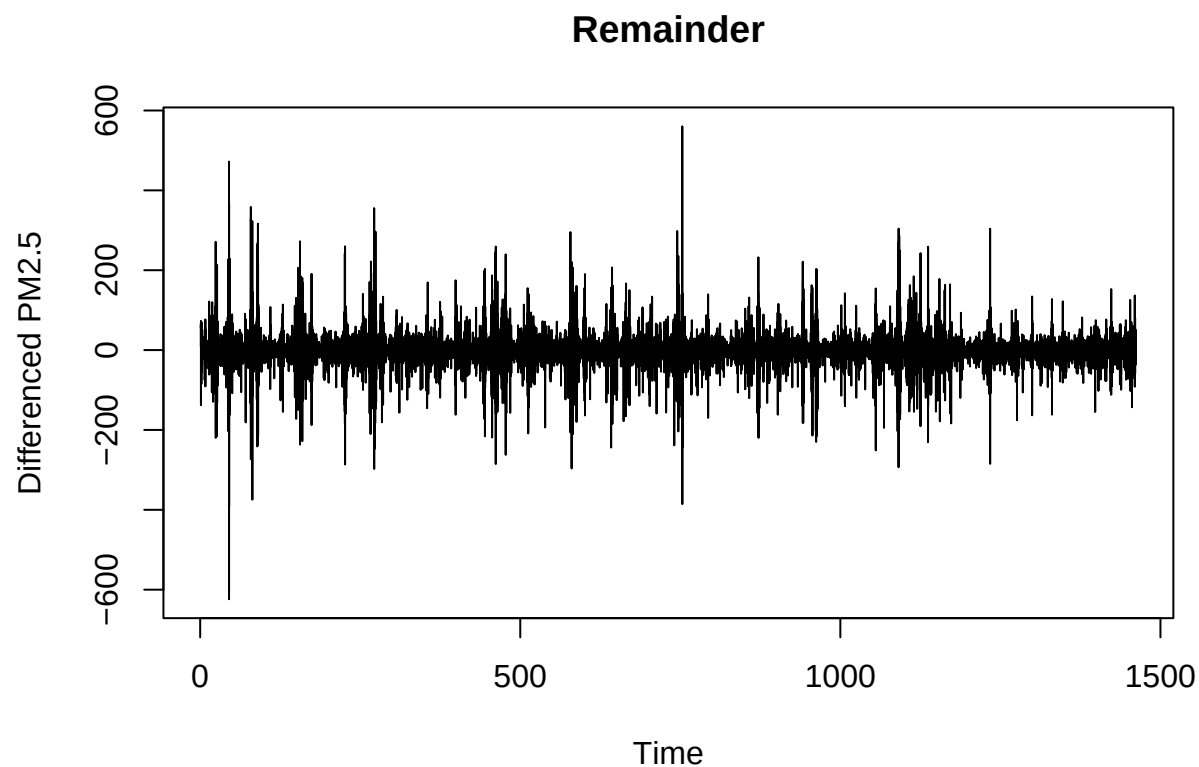


```
plot(diff_decomp$time.series[, "seasonal"], ylab = "Differenced PM2.5", main = "Seasonal")
```

Seasonal



```
plot(diff_decomp$time.series[, "remainder"], ylab = "Differenced PM2.5", main = "Remainder")
```



Similar to the original time-series, the differenced time-series also doesn't show any trend or seasonality, with the residuals somewhat resembling white noise while still indicating the presence of potential outliers and unexplained events.

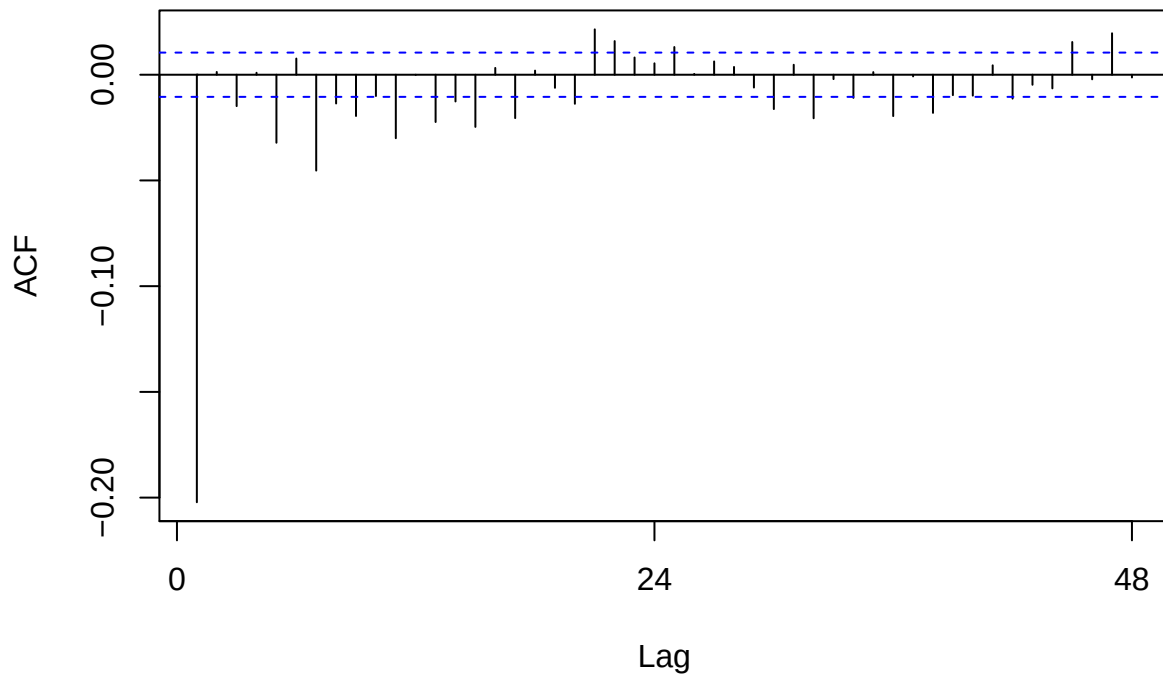
```
# Re-check stationarity  
# ADF test  
adf.test(diff_pm25)
```

```
## Warning in adf.test(diff_pm25): p-value smaller than printed p-value
```

```
##  
## Augmented Dickey-Fuller Test  
##  
## data: diff_pm25  
## Dickey-Fuller = -40.059, Lag order = 32, p-value = 0.01  
## alternative hypothesis: stationary
```

```
# Plot ACF  
Acf(diff_pm25)
```

Series diff_pm25



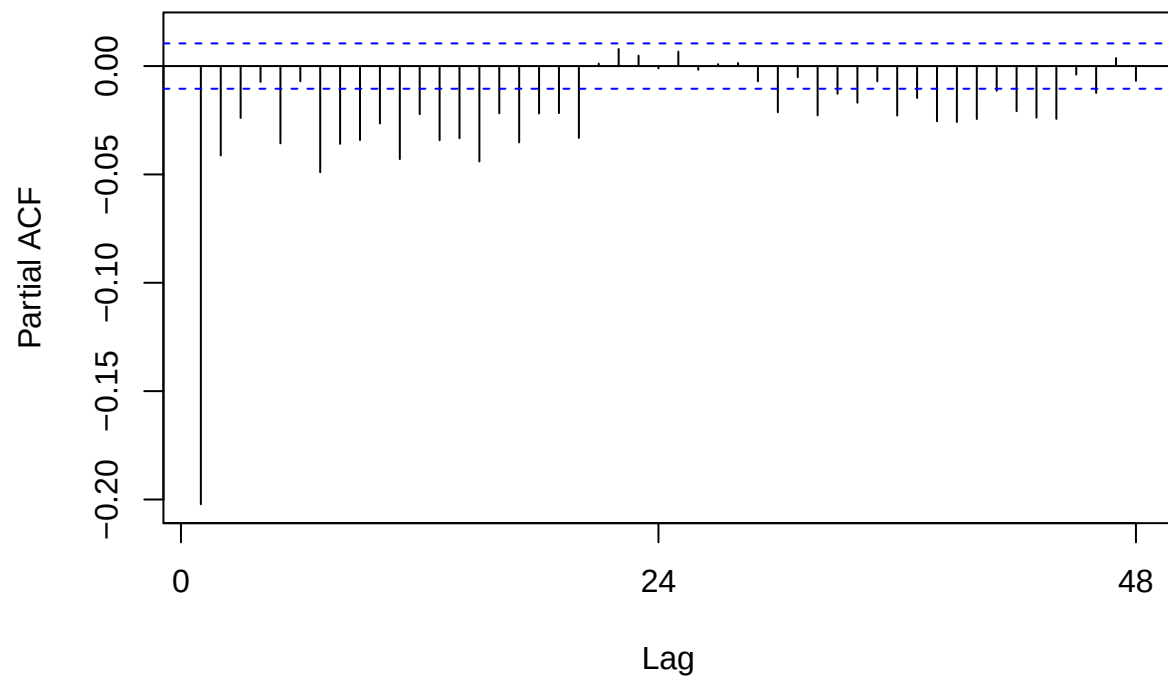
```
# If there is non-stationarity related to seasonal pattern(s), you may instead / also  
# need to use 1st order seasonal differencing (subtracting the observation that occurred  
# one seasonal period ago from the current observation) to remove seasonality  
# (via D=1 and s=24 in SARIMA/SARIMAX)  
# ss_diff_pm25 <- diff(diff_pm25, lag = 24, differences = 1)  
  
# If there is seasonal pattern(s) but no non-stationarity, you may need seasonal AR and  
# MA order (P and Q), even with D = 0
```

Since the p-value of the ADF test is smaller than 0.01, the null hypothesis can be rejected at the 0.01 significance level. This indicates that the differenced time-series is stationary. In addition, according to the ACF plot, the ACF values drop close to 0 quickly only after the first 2 lags, suggesting stationarity.

Although the differenced time-series appears to be stationary with a constant mean of 0, not all lags in the ACF plot are insignificant, suggesting that the series isn't white noise. Therefore, the differenced time-series is used for modeling ($d = 1$).

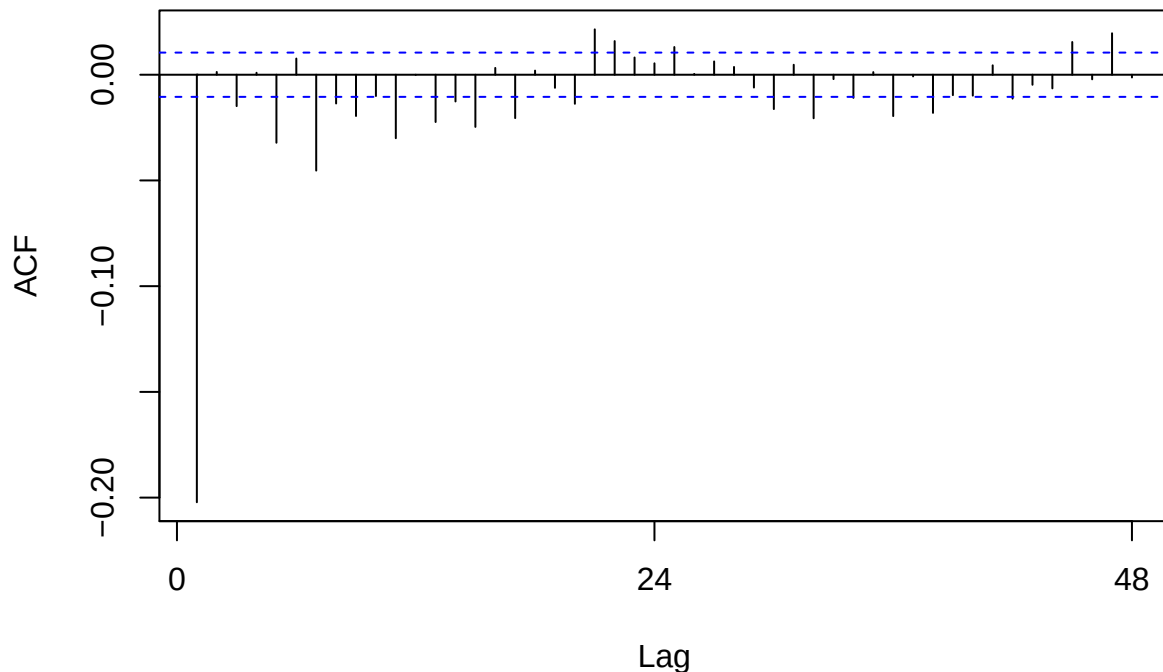
```
# Determine the AR and MA order for ARIMAX  
Pacf(diff_pm25)
```

Series diff_pm25



```
Acf(diff_pm25)
```

Series diff_pm25



*# In ARIMA/ARIMAX, the AR and MA orders (p and q) are identified using the PACF and ACF plots.
 # Specifically, the order is determined by observing the number of initial lags that show
 # significant spikes before the correlations become insignificant or diminished
 # (similar to a cutoff value)*

*# In SARIMA/SARIMAX, the seasonal AR and MA orders (P and Q) are also identified using
 # the PACF and ACF plots. Specifically, the order is determined by observing the number of
 # initial seasonal lag multiples (only observe lag = 24, 48, 72, ... if s = 24) that show
 # significant spikes before the correlations become insignificant or diminished*

*# ACF/PACF are only approximate guides. In practice, people still often try nearby simpler models
 # as well for comparison. Simpler models are preferred if performance is comparable*

The PACF and ACF plots suggest that the AR and MA orders are $p = 1$ and $q = 2$, respectively. Therefore, the initial model would be ARIMAX($p=1$, $d=1$, $q=2$)

```
xreg_train <- train_scaled[, c("dewp", "temp", "pres", "cbwd", "iws", "is", "ir")]

# Convert any external factors / categorical variables into dummy variables
xreg_train <- dummy_cols(xreg_train, select_columns = "cbwd",
                        remove_most_frequent_dummy = TRUE,
                        remove_selected_columns = TRUE)
xreg_train <- as.matrix(xreg_train)

# Fit the ARIMAX model
arimax_mod <- Arima(train_scaled$pm25, order = c(1, 1, 2),
                    seasonal = list(order = c(0L, 0L, 0L), period = NA),
```

```

xreg = xreg_train) # No seasonality

# Apply the model to the original time series instead of the differenced time series,
# since the differencing is done inside the model.

# Use auto.arima() instead for automatic estimation of parameters (p,d,q,P,D,Q)

# # Fit the ARIMAX model
# arimax_mod <- auto.arima(train_scaled$pm25, stationary = FALSE, seasonal = TRUE,
#                           xreg = xreg_train, trace = TRUE)
# # stationary = FALSE to enable non-stationary options, seasonal = TRUE to enable seasonality search

y_train <- train_scaled$pm25
y_test  <- test_scaled$pm25

mean(y_train)

## [1] 98.78476

sd(y_train)

## [1] 91.4202

```

The fact that the standard deviation of the training set's time-series is almost as large as the mean shows that the PM25 concentration time-series is extremely volatile and noisy.

```

summary(arimax_mod)

## Series: train_scaled$pm25
## Regression with ARIMA(1,1,2) errors
##
## Coefficients:
##          ar1      ma1      ma2      dewp      temp      pres      iws      is
##          0.9294 -1.2051  0.2084  56.7980 -14.2320 -14.3182 -2.8089 -0.0769
## s.e.      0.0025  0.0059  0.0058   2.0029   1.4673   2.2886   0.5819   0.4084
##          ir  cbwd_cv  cbwd_NE  cbwd_NW
##          -2.0238 -0.6655 -3.6923 -1.3977
## s.e.      0.4145   0.5309   0.7130   0.7030
##
## sigma^2 = 1163: log likelihood = -173472.6
## AIC=346971.2  AICc=346971.2  BIC=347081.3
##
## Training set error measures:
##              ME      RMSE      MAE MPE MAPE      MASE      ACF1
## Training set 0.01078266 34.09614 18.05174 Inf  Inf  0.2549911 -0.001408841

```

(The training set error measures like MAE/RMSE are computed using one-step forecasting which only updates the internal state at every step)

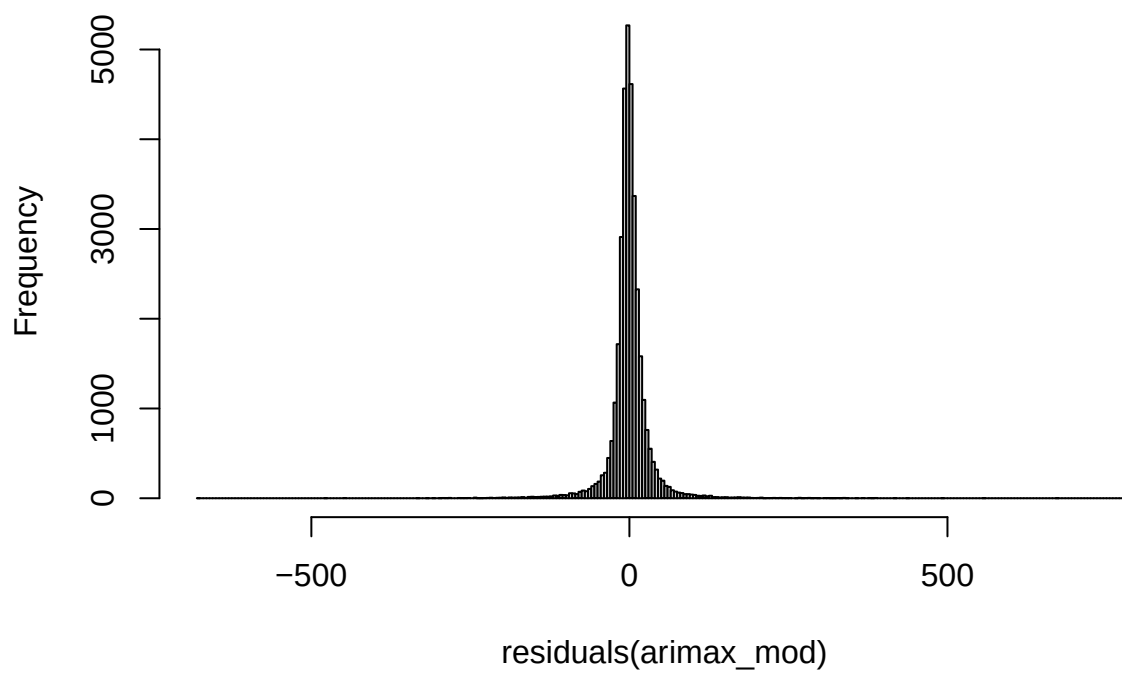
The MAE value (18.05) is approximately 18.3% of the mean PM2.5 concentration (98.78). Moreover, the RMSE value (34.10) is approximately 37.3% of the standard deviation of the series (91.42). These results indicate a good model fit. Furthermore, the MASE value of $0.25 < 0.5$ suggests that the model substantially outperforms a naive forecasting model benchmark on the training data.

```

# Check if residuals are independently distributed (behave like white noise)
hist(residuals(arimax_mod), breaks = "Scott") # Breaks affect bin size

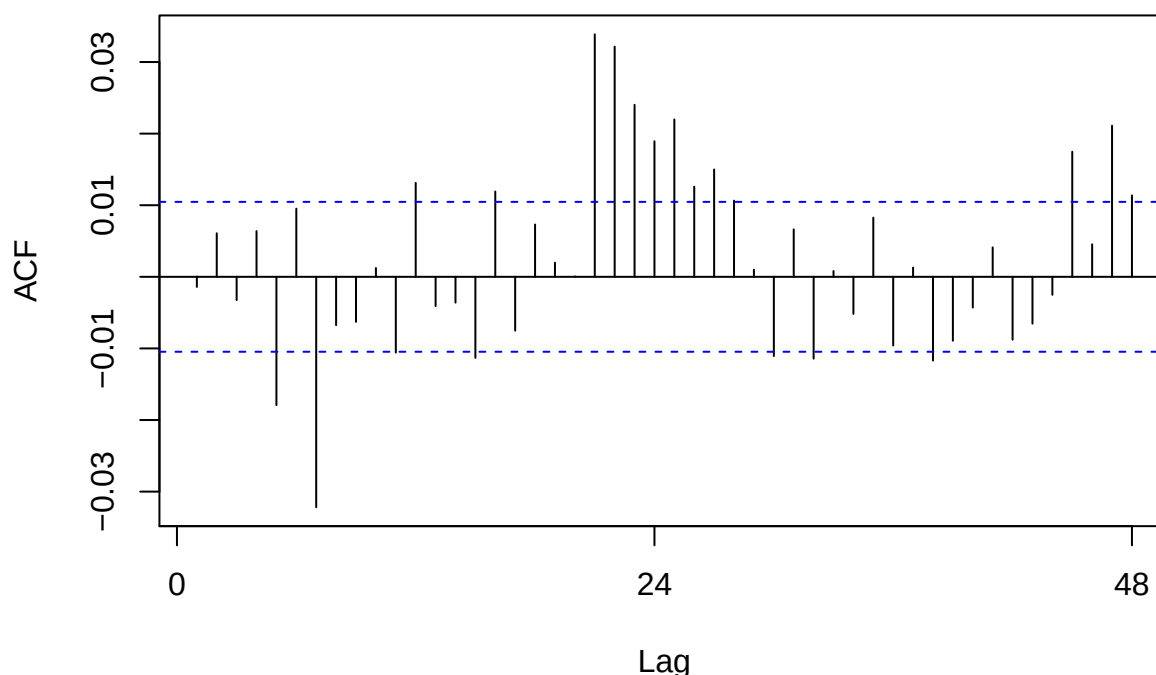
```

Histogram of residuals(arimax_mod)



```
Acf(residuals(arimax_mod))
```

Series residuals(arimax_mod)



```
# Ljung-Box test
Box.test(residuals(arimax_mod), lag = 24, type = "Ljung-Box")

##
## Box-Ljung test
##
## data: residuals(arimax_mod)
## X-squared = 190.76, df = 24, p-value < 2.2e-16
# This tests whether the residual autocorrelations at lag = 1,...,24 are jointly insignificant.
# Here lag isn't seasonal period

# Residuals behaving like white noise means the model has captured most of the data structure
```

The residual histogram is centered around zero. Moreover, the residual ACF plot shows that most autocorrelation has been removed. However, several small but statistically significant autocorrelations remain at various lags. This observation is further supported by the Ljung–Box test up to lag 24 ($p < 2.2 \times 10^{-16} < 0.01$), which rejects the null hypothesis of independently distributed residuals. Nevertheless, the magnitudes of the residual autocorrelations are generally small (approximately 0.03), indicating that the remaining dependence is relatively weak.

Overall, the diagnostics suggest that the ARIMAX model captures the major structure of the PM2.5 series, although some residual autocorrelation remains unaccounted for.

```
## Multi-step forecasting on test set (shouldn't be used on training set)
xreg_test <- test_scaled[, c("dewp", "temp", "pres", "cbwd", "iws", "is", "ir")]

# Convert any external factors / categorical variables into dummy variables
```

```

xreg_test <- dummy_cols(xreg_test, select_columns = "cbwd",
                        remove_most_frequent_dummy = TRUE,
                        remove_selected_columns = TRUE)
xreg_test <- as.matrix(xreg_test)

test_pred_multi <- forecast(arimax_mod, h = length(test_scaled$pm25), xreg = xreg_test)$mean
# h is number of periods for forecasting, mean is the point forecast

# Evaluate model on test set based on multi-step forecasting result
rmse(y_test, as.numeric(test_pred_multi))

## [1] 121.3189

mae(y_test, as.numeric(test_pred_multi))

## [1] 102.6511

```

The multi-step forecasting evaluation yielded an RMSE of 121.32 and an MAE of 102.65 on the test set, indicating substantially worse forecasting performance than observed during model fitting. As forecasting errors tend to accumulate over longer forecasting horizons, additional evaluations using m-step forecasting approaches are conducted to assess whether more frequent model updates can improve predictive performance.

Suppose ARIMAX forecasting starts after time T . In multi-step forecasting, the model is fitted once using the observed values up to time point T and is not updated thereafter. Multi-step forecasting uses the real observations up to time T , as well as the predicted values of time points $T+1, \dots, T+k$, in order to forecast the observation of time point $T+k+1$. In contrast, one-step forecasting only uses the real observations up to time point $T+k$ in order to forecast the observation of time point $T+k+1$, with the model being updated after each newly observed value becomes available. Since real observations can only be revealed one by one in time, one-step forecasting is more time consuming, but tends to be more accurate.

(In other ML/DL models, multi-step forecasting can have different meanings (direct/multi-output) from ARIMA-type models)

As a compromise between these two approaches, h -step forecasting ($h > 1$) uses all real observations up to time T to forecast the observations at time points $T+1, \dots, T+h$. Once the actual observations for these m time periods become available, the model is updated and used to forecast the next h observations. Therefore, h -step forecasting generally produces more accurate forecasts while requiring less computational effort than one-step forecasting.

During one-step (or h -step) forecasting for ARIMAX, it's possible to refit the model at every step (or every h steps) or just update the internal state (*Kalman filter*) instead to reduce runtime. It's also possible to use either a moving / rolling training window (*requires a new window size hyperparameter*) or an expanding training window if you refit the model at every step (or every h steps). In this project, 24-step forecasting is used, with only the internal state being updated every 24 steps.

(Updating the internal state means the model keeps its existing, previously calculated ARIMA coefficients, and only uses the new real observation to compute the new residual and shifts the lag windows forward. Moving training window means the size of your training data stays fixed. As a new real observation is added at the end, the oldest observation may need to be dropped from the beginning)

In order to compare the model performance on both the training and test set, one-step / h -step forecasting would be more suitable.

(In ARIMA, multi-step forecasting would return the same prediction on time $T+k$ for both the training and test set. In ARIMAX, this wouldn't happen because of the difference in xreg matrix)

```

## 24-step forecasting on training and test set which only updates the internal state every 24 steps

# On training set

```

```

train_pred_one <- numeric(length(y_train))
n <- length(y_train)
h <- 24 # h=24: Model is updated every 24 hours = 1 day
i <- 1

history_y <- y_train
history_x <- xreg_train

tic()

while (i <= n) {
  h_eff <- min(h, n - i + 1)
  # Effective forecast horizon, to handle the last block of the training set safely

  fit <- Arima(history_y, xreg = history_x, model = arimax_mod)
  # Use model = arimax_mod to only update the internal state and not refit the model

  train_pred_one[i:(i + h_eff - 1)] <-
    as.numeric(forecast(fit, h = h_eff,
                        xreg = xreg_train[i:(i + h_eff - 1), , drop = FALSE])$mean)

  # Reveal the true observation
  history_y <- c(history_y, y_train[i:(i + h_eff - 1)])
  history_x <- rbind(history_x, xreg_train[i:(i + h_eff - 1), , drop = FALSE])

  i <- i + h_eff
}

toc() # Print runtime

```

```
## 168.3 sec elapsed
```

```

# On test set
test_pred_one <- numeric(length(y_test))
n <- length(y_test)
h <- 24
i <- 1

history_y <- y_train
history_x <- xreg_train

tic()

while (i <= n) {
  h_eff <- min(h, n - i + 1)
  # Effective forecast horizon, to handle the last block of the test set safely

  fit <- Arima(history_y, xreg = history_x, model = arimax_mod)

  test_pred_one[i:(i + h_eff - 1)] <-
    as.numeric(forecast(fit, h = h_eff,
                        xreg = xreg_test[i:(i + h_eff - 1), , drop = FALSE])$mean)

  # Reveal the true observation

```

```

history_y <- c(history_y, y_test[i:(i + h_eff - 1)])
history_x <- rbind(history_x, xreg_test[i:(i + h_eff - 1), , drop = FALSE])

i <- i + h_eff
}

toc() # Print runtime

```

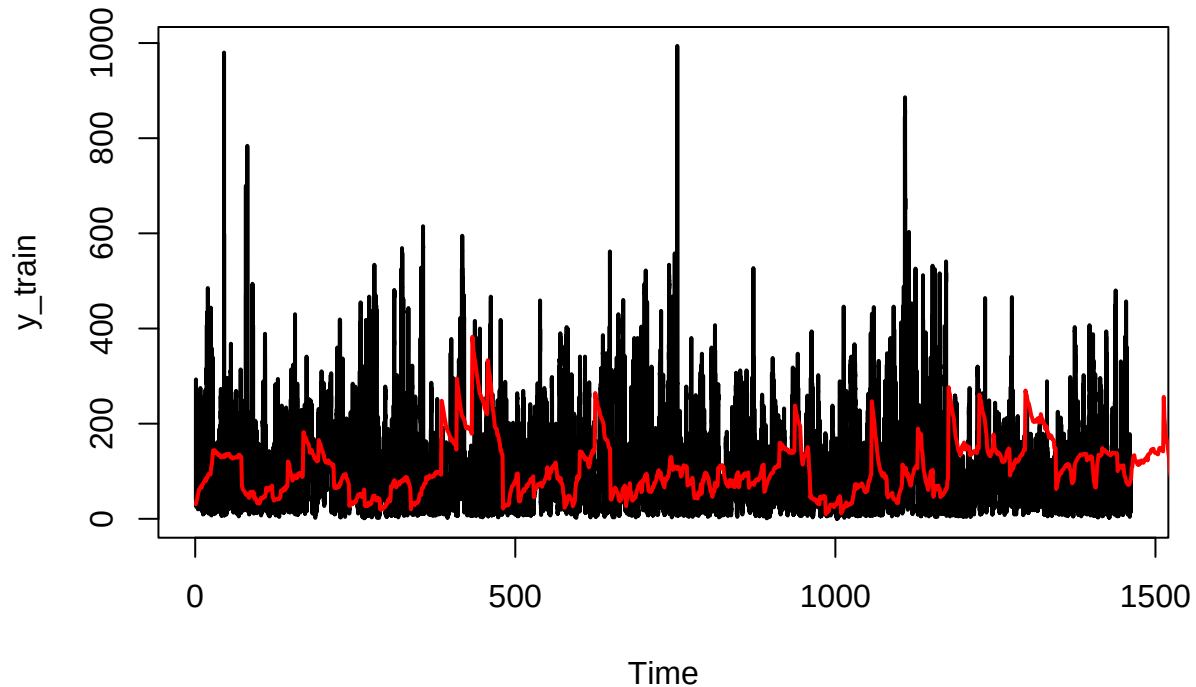
39.2 sec elapsed

Here, smaller h values (like $h=1,6,12$) isn't used since it is not realistic to update the model every few hours. When the forecasting horizon is too short, the forecasting problem will be less meaningful and will resemble more like a tracking task.

```

plot(y_train, type = "l", col = "black", lwd = 2)
lines(train_pred_one, col = "red", lwd = 2)

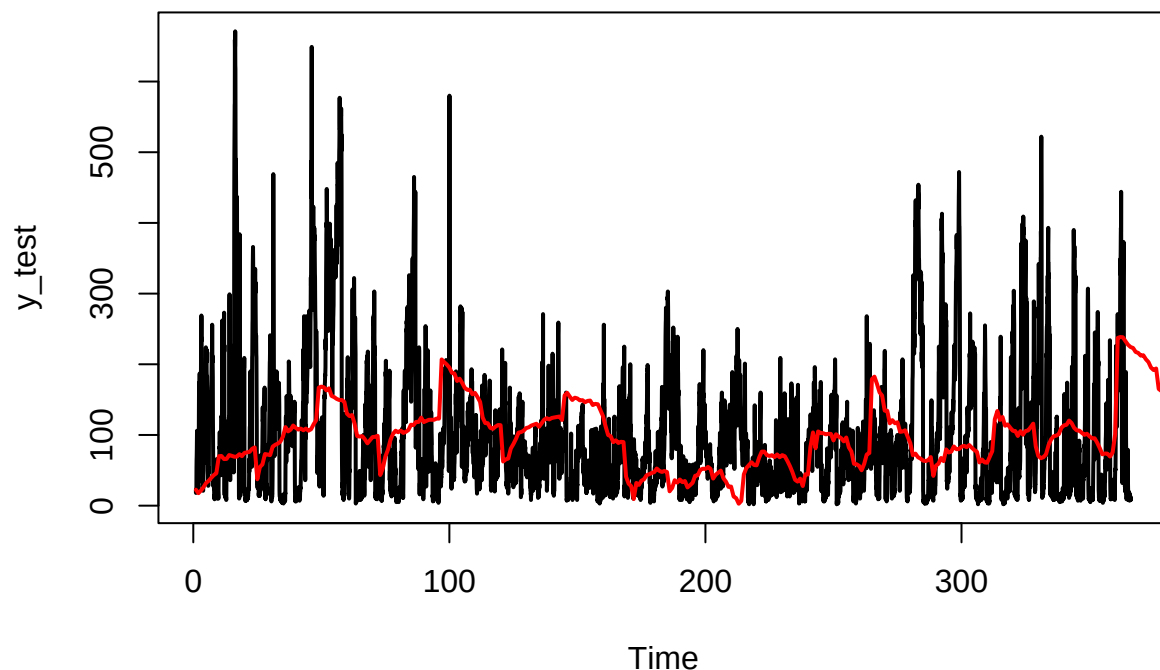
```



```

plot(y_test, type = "l", col = "black", lwd = 2)
lines(test_pred_one, col = "red", lwd = 2)

```



In both the training and test sets, the forecast (red line) mainly captures the overall movement of the series but fails to follow most of the short-term high fluctuations. Overall, this suggests the model is reasonable for capturing broad patterns in the data, but not good enough for capturing short-term volatility or extreme events.

```
# Evaluate model on training and test set based on 24-step forecasting result
```

```
# On training set
```

```
rmse(y_train, train_pred_one)
```

```
## [1] 65.54727
```

```
mae(y_train, train_pred_one)
```

```
## [1] 44.79942
```

```
# On test set
```

```
rmse(y_test, test_pred_one)
```

```
## [1] 60.21544
```

```
mae(y_test, test_pred_one)
```

```
## [1] 41.88462
```

On the 24-step forecasting setup, there is only a small difference between the error measures on the training and test sets, indicating stable generalization performance. Notably, the test errors are even slightly lower than the training errors, which suggests that the model is not overfitting and that this difference in performance may be explained by differences in the data characteristics between the training and test periods.

In particular, the fact that the model performs better on the test set than on the training set is most likely due to the training period containing more outliers (noise spikes) than the test period, as observed in the time-series plots above. This makes the test set somewhat easier to forecast than the training set.

The test MAE value (41.88) is about 42.4% of the mean PM2.5 concentration in the training data (98.78), while the test RMSE value (60.22) is about 65.9% of the training standard deviation (91.42). This suggests that although the model captures a substantial portion of the overall variability, it still leaves a large fraction of short-term fluctuations unexplained. However, these results are still much better than the results on the multi-step forecasting setup above (RMSE of 121.32 and MAE of 102.65).

```
# Compare model performance to a naive forecasting model with step_size = 1, ..., h
# (y_{t} = y_{t-stepsize})
mase(y_train, train_pred_one, step_size = 1)
```

```
## [1] 2.657376
```

```
mase(y_test, test_pred_one, step_size = 1)
```

```
## [1] 3.325513
```

```
mase(y_train, train_pred_one, step_size = 6)
```

```
## [1] 1.035718
```

```
mase(y_test, test_pred_one, step_size = 6)
```

```
## [1] 1.066649
```

```
mase(y_train, train_pred_one, step_size = 12)
```

```
## [1] 0.7698837
```

```
mase(y_test, test_pred_one, step_size = 12)
```

```
## [1] 0.7710474
```

```
mase(y_train, train_pred_one, step_size = 24)
```

```
## [1] 0.6328176
```

```
mase(y_test, test_pred_one, step_size = 24)
```

```
## [1] 0.6176962
```

```
# h-step forecasting: Forecast the next 24 observations at a time, so the
# forecasting horizon changes from 1 to 24
```

The MASE results show a clear dependence on the choice of step size. For short horizons, the model performs worse than a naive forecasting benchmark, with MASE values above 1 for step sizes 1 and 6. This indicates that the model is not as effective at capturing short-term dynamics as a naive forecasting model. However, performance improves as the step size increases. At step size 12 and 24, the MASE values drop below 1, indicating that the model outperforms the naive forecasting benchmark for longer-term forecasting.

Overall, the results indicate that the model, on the 24-step forecasting setup, is less effective relative to a naive forecasting benchmark at short forecasting horizons, but becomes increasingly competitive as the horizon increases. This is not surprising, as naive forecasting models are typically very strong at short horizons.

Therefore, the ARIMAX(1,1,2) model is considered suitable for predicting the PM2.5 concentration time-series, using the 24-step forecasting strategy which only updates the internal state at every 24 steps.

Compared to ARIMA, ARIMAX has the limitation of requiring future values of the external predictors to be known. In order to forecast future PM2.5 concentration values, the values of the external weather variables can be obtained from weather forecasts.